

Intermediate Data Structures & Algorithms

CSCI 232

Fall 2026

Anh Nguyen, Ph.D.

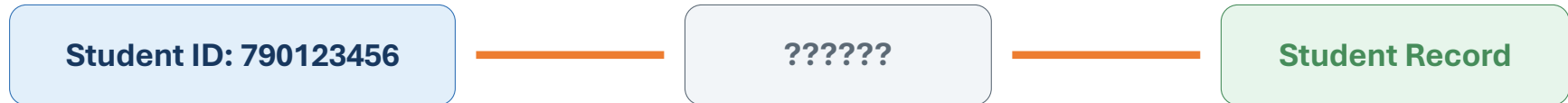
Assistant Professor of CS

The University of Montana

Maps, Hash Tables, and Skip Lists

How Do We Find a Record Efficiently?

Suppose we store 20,000 student records.



- ☐ Scan records one by one
- ☐ Keep records sorted
- ☐ Jump directly to the desired location

What Is a Map?

❑ A map stores key-value pairs.

United States	→	Dollar
India	→	Rupee
China	→	Yuan
Spain	→	Euro
Greece	→	Euro

- Can two keys have the same value?
- Can one key appear twice?

- Each key is unique
- Values do not have to be unique
- A key identifies its associated value

Map vs. Array/List

ARRAY / LIST

0	1	2
90	85	97

A[1] → 85

MAP

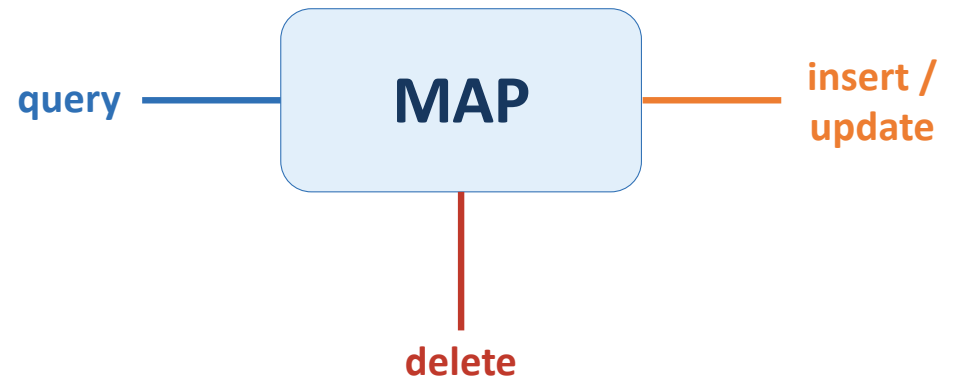
Alice	→	90
Bob	→	85
Carol	→	97

```
A = [90, 85, 97]  
print(A[1])
```

```
M = {'Alice': 90, 'Bob': 85, 'Carol': 97}  
print(M['Bob'])
```

Which representation is more natural when retrieving a grade by student name?

Map ADT



Five core operations:

- $M[k]$ # retrieve value
- $M[k] = v$ # insert or update
- $\text{del } M[k]$ # delete
- $\text{len}(M)$ # number of entries
- $\text{iter}(M)$ # iterate over keys

Map ADT

Other useful map operations:

- ☐ `k in M`
- ☐ `M.get(k, default)`
- ☐ `M.setdefault(k, default)`
- ☐ `M.pop(k)`
- ☐ `M.keys()`
- ☐ `M.values()`
- ☐ `M.items()`
- ☐ `M.update(M2)`

```
M = {'K': 9, 'B': 4, 'U': 2}
```

```
print(M.get('K', 0))    # 9  
print(M.get('X', 0))    # 0
```

Map ADT vs. Priority Queue ADT

- ❑ Both can store (key, value) pairs
- ❑ BUT the key serves a different purpose.

	MAP ADT	PRIORITY QUEUE ADT
Meaning of key	Identifier	Priority
Main question	What value belongs to key k?	Which item comes next?
Primary access	$M[k]$	$\text{min}() / \text{max}()$
Removal	Specified key	Best-priority item
Duplicate keys	No — update existing association	Yes — equal priorities are valid
Ordering by key	Not required	Fundamental to behavior

Map ADT vs. Priority Queue ADT

❑ Which ADT do the following applications need?

1 Students' record retrieval

MAP

Need lookup by a specific identifier.

2 Hospital triage

PRIORITY QUEUE

Need the most urgent patient next.

3 User's profile searching

MAP

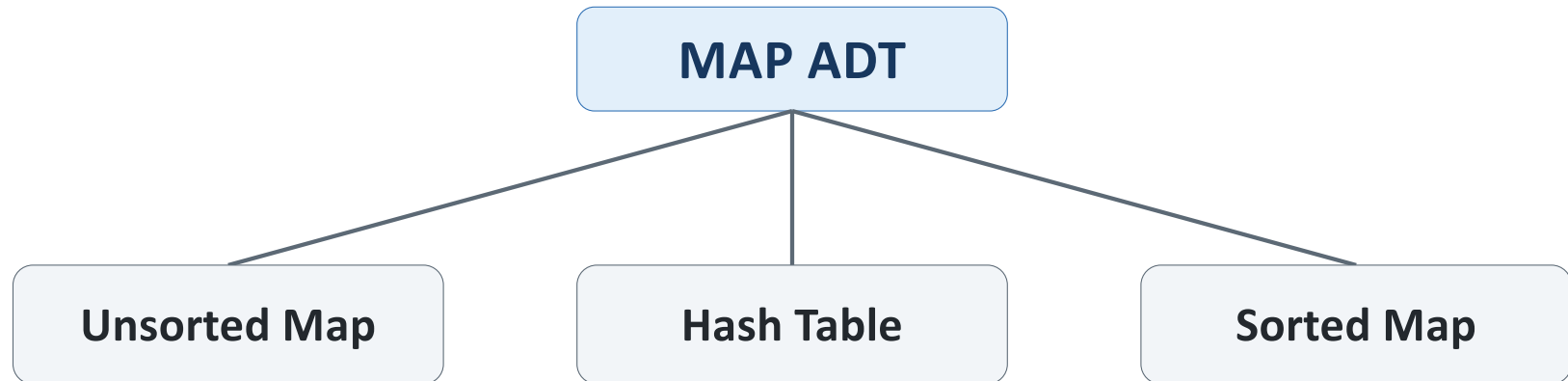
Need direct lookup by username.

4 Printer's jobs

PRIORITY QUEUE

Need the highest-priority job next.

Map Implementation



Simplest Implementation: Unsorted Map

❑ Internal representation

$[(K, 9), (B, 4), (U, 2), (V, 8)]$

○ Search for U

$(K, 9) \rightarrow (B, 4) \rightarrow (U, 2) \rightarrow \text{FOUND}$

❑ What happens if the key is not present?

Simplest Implementation: Unsorted Map

Operation	Worst case
Search	$O(n)$
Insert/update	$O(n)$
Delete	$O(n)$
len()	$O(1)$
Iteration	$O(n)$

Can we make lookup close to $O(1)$?

What If Keys Were Array Indices?

0	1	2	3	4	5	6	7	8	9	10
	D		Z			C	Q			

```
value = table[k]
```

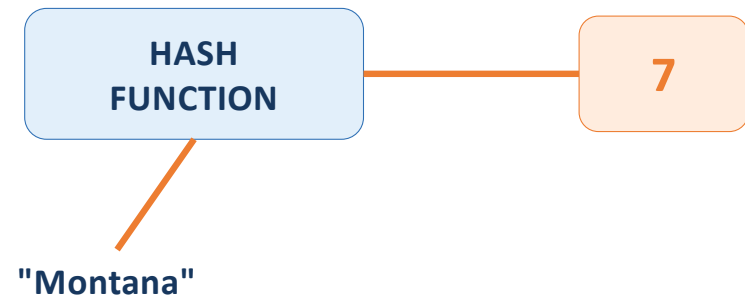
Direct array access is $O(1)$.

➔ What's wrong with this approach for keys such as 'Montana'?

Hash Tables

- ❑ A data structure that stores key-value pairs and allows for very fast insertion, deletion, and search operations in typical applications
- ❑ Hash function
 - Map general keys to corresponding indices in a table

"Montana"	_____	?
"CSCI232"	_____	?
"student42"	_____	?
"apple"	_____	?



Hash Functions

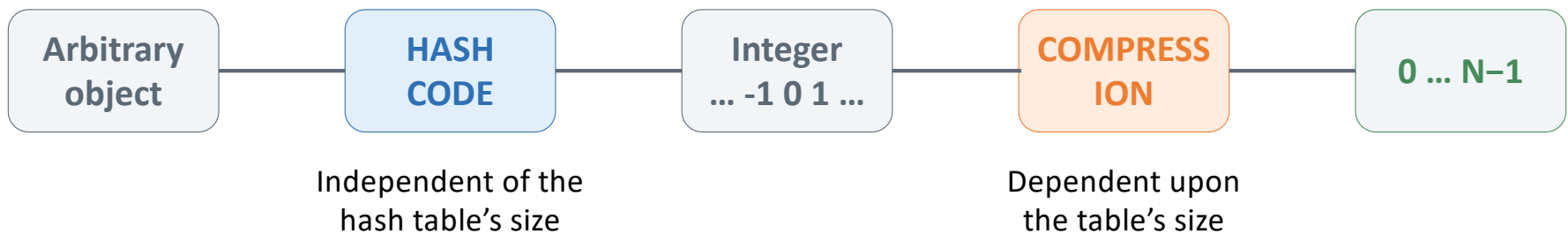
- ❑ For a table of capacity N :

$$h(k) \in [0, N - 1]$$



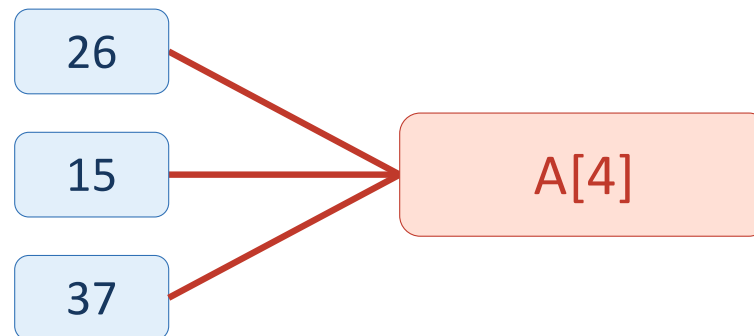
- ❑ Store the item (k, v) in the table $A[h(k)]$

- ❑ Two steps:



Hash Functions

- ❑ Implementing a *good* hash function depends on the key type.
- ❑ Collision = DIFFERENT keys, SAME hash value (i.e., table index)



- Collision handling schemes

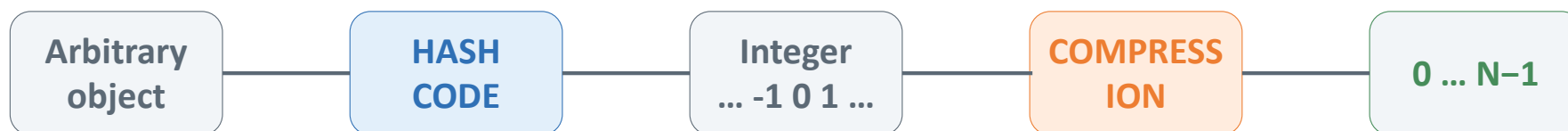
- Provide a strategy for handling the case when two or more keys to be inserted hash to the same index
- Two primary approaches:
 - Chaining (e.g., separate chaining) and Open addressing (e.g., linear probing)

Hash Functions

□ What properties would you want from a good hash function?

- *Consistent*
 - i.e., equal keys must produce the same hash value.
- *Efficient to compute*
- *Uniformly distribute* the keys
 - i.e., for each key, every integer between 0 and $M - 1$ should be equally likely (independently for every key).
- Deterministic
- Minimizes collisions

Hash Functions



□ Hash codes

1. Bit representation as an Integer
2. Polynomial hash codes
3. Cyclic-shift hash codes

Hash Functions

□ Hash codes

1. Bit representation as an Integer

- If the key fits within the hash-code width, interpret its bits directly as an integer.
- Example: integer key 314 can simply use hash code 314.
- For a longer representation, combine multiple fixed-width portions using addition or XOR so that information from all bits contributes.

high 32 bits



low 32 bits

- Limitation?
 - Using only half the bits can create many collisions.

2. Polynomial hash codes

3. Cyclic-shift hash codes

Hash Functions

□ Hash codes

1. Bit representation as an Integer

- How about strings or variable-length objects?

stop

tops

pots

spot

- Can we merely sum character values?
 - Lose positional information if order matters
 - All contain the same characters → the same simple sum → collision
- We need a hash code that depends on both:
 - Character values
 - Character positions

2. Polynomial hash codes

3. Cyclic-shift hash codes

Hash Functions

□ Hash codes

1. Bit representation as an Integer

2. Polynomial hash codes

- Give each component a position-dependent weight using powers of a nonzero constant a .

$$x_0a^{n-1} + x_1a^{n-2} + \cdots + x_{n-2}a + x_{n-1}$$

- Horner's rule computes the same polynomial efficiently:

$$x_{n-1} + a(x_{n-2} + a(x_{n-3} + \cdots + a(x_1 + ax_0)))$$

- E.g., $s = \text{"call"}$

$$h = 3045982$$

$$= 99 \cdot 31^3 + 97 \cdot 31^2 + 108 \cdot 31^1 + 108 \cdot 31^0$$

Order matters

Good spread

Fast via Horner

3. Cyclic-shift hash codes

Hash Functions

□ Hash codes

1. Bit representation as an Integer
2. Polynomial hash codes
3. Cyclic-shift hash codes
 - Replace multiplication by a cyclic shift of the running hash value.
 - Example: 5-bit cyclic left shift of a 32-bit running value

00111101100101101010100010101000



1011001011010101010001010100000111

- Then add the value of the next character.

Hash Functions

□ Hash codes

Strategy	Core idea	Best suited / key lesson
Bit representation	Interpret/combine bits	Simple fixed-width values; use all bits
Polynomial	Position-weighted components	Strings/sequences where order matters
Cyclic shift	Rotate running bits + add next component	Order-sensitive strings without multiplication

Compression Functions

❑ Division method

- Maps an integer i to

$$i \bmod N$$

where N is

- the hash table's size
- usually a prime number

❑ Multiply-Add-and-Divide (MAD) method

- Maps an integer i to

$$[(ai + b) \bmod p] \bmod N$$

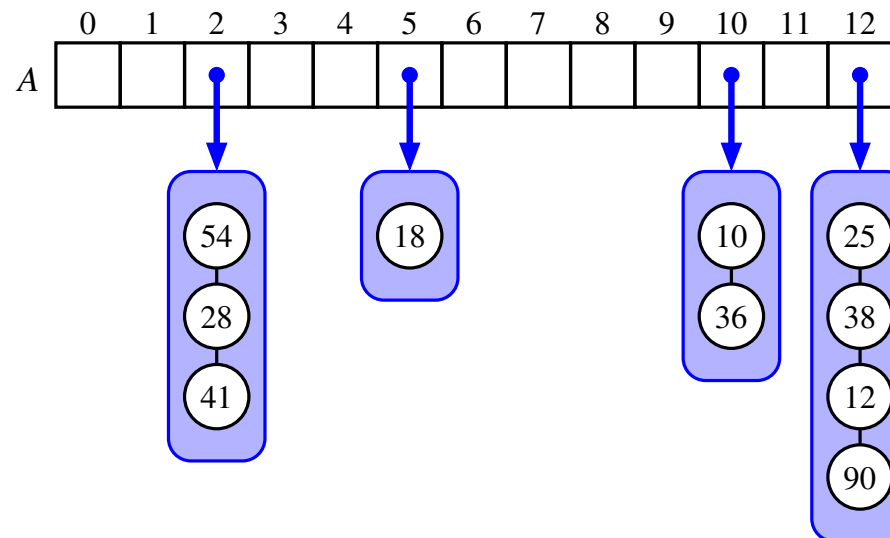
where

- p : a prime number larger than N
- a, b : integers chosen randomly from $[0, p - 1]$ with $a > 0$

Collision Handling

❑ Separate chaining

- Each bucket stores a secondary collection of entries.



- If bucket 2 contains three entries, how do we find key 28?

Collision Handling

□ Separate chaining

- Load factor

$$\lambda = n / N$$

where

- n = number of entries
- N = number of buckets

- Example:

- $n = 8, N = 11$
- $\lambda = 8/11 \approx 0.73$

- If $\lambda = 0$,

Collision Handling

□ Separate chaining

- Low λ (e.g., <1)
 - Too many empty buckets
 - Fewer collisions
 - Faster lookups but more wasted space.
- High λ (e.g., >1)
 - Buckets are too long
 - More collisions (multiple keys hashing to the same bucket)
 - Slower lookups and insertions.
- For chaining, $\lambda \approx$ average entries per bucket and is generally > 1

Collision Handling

□ Separate chaining

- Choose N to be sufficiently large so that the buckets are sufficiently short to enable efficient search through a two-step process:
 - Hash to find the list that could contain the key, then
 - Sequentially search through that list for the key.
- Typical choice: $N \sim n / 5 \Rightarrow$ constant-time ops
- Rehashing (resizing) often occurs when λ exceeds a threshold like 1.0 or 2.0.
- Key order is not important!

Collision Handling

❑ Open addressing

- Store every item directly inside the table.

				probe 1	probe 2	probe 3	probe 4 → INSERT
0	1	2	3	4	5	6	7
				26	37	16	

- Insert key 15 (mod 11)?

$$h(15) = 4$$

A[4] occupied!

- Linear probing

If $A[h(k)]$ is occupied, try the next slot, then the next...

Collision Handling

□ Linear probing

○ Searching

0	1	2	3	4	5	6	7
				26	37	16	

Search for 16: $h(16) = 5$

5	6
37	16

FOUND ✓

Search for 15: $h(15) = 4$

4	5	6	7
26	37	16	

NOT FOUND ✗

- When can we stop searching?
Keep probing until the key is found
OR a truly EMPTY slot appears.

- Key equals search key: search hit
- Empty position (null key at indexed position): search miss
- Key not equal to search key: try next entry

Collision Handling

❑ Linear probing

○ Deletion

Before deletion

4	5	6	7
26	37	16	15

Naively delete 37

4	5	6	7
26		16	15

- Search for 15?

4 → 5

EMPTY → STOP

→ Incorrectly conclude: 15 is absent

- How could we distinguish *never used* from *previously occupied*?

Collision Handling

□ Linear probing

○ Deletion

- Tombstones / **AVAILABLE** Marker



Continue searching across AVAILABLE

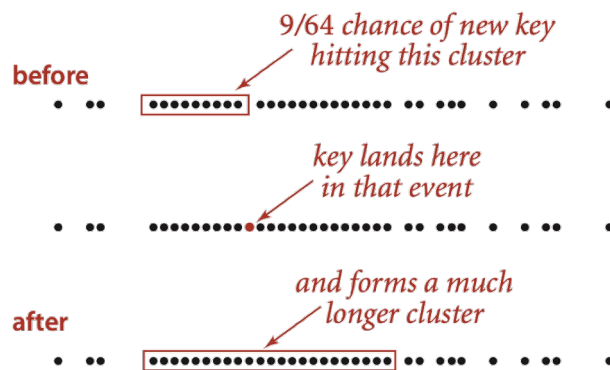
- EMPTY → stop search
- AVAILABLE → continue search

○ Can a new key be inserted into an AVAILABLE location?

Collision Handling

□ Linear probing

- *Probing* = the operation of determining whether or not a given table entry holds an item whose key is equal to the search key
- *Clustering*: the way in which the entries clump together into contiguous groups of occupied table entries, called clusters, when they are inserted.
 - Short clusters are certainly a requirement for efficient performance.



Collision Handling

❑ Linear probing

- Good distribution



- Primary clustering



- More probes
- Slower searches
- More collisions during insertion

Collision Handling

□ Alternatives to linear probing

- Linear probing

- $4 \rightarrow 5 \rightarrow 6 \rightarrow \dots$

$$h(k) + i$$

- Quadratic probing

- $4 \rightarrow 5 \rightarrow 8 \rightarrow \dots$

$$h(k) + i^2$$

- Double hashing

- $4 \rightarrow 9 \rightarrow 3 \rightarrow \dots$

$$h(k) + i \cdot h'(k)$$

Collision Handling

□ Linear probing

- Load factor

$$\lambda = n / N$$

where

- n = number of entries
- N = hash table's size

○ Examples:

- $\lambda \approx 0.25$

X				X			X		
---	--	--	--	---	--	--	---	--	--

- $\lambda \approx 0.90$

X	X	X	X	X	X	X	X	X	
---	---	---	---	---	---	---	---	---	--

Hash Tables Performance

❑ What happens when the table gets too full?

○ Old table

x	x	x		x
---	---	---	--	---

REHASH
not copy

○ Allocate a larger table

--	--	--	--	--	--	--	--	--	--	--

○ Reinsert / rehash every entry

x			x		x			x		
---	--	--	---	--	---	--	--	---	--	--

Hash Tables Performance

❑ Isn't rehashing expensive?

- One resize

$O(n)$

- But resizing occurs only occasionally.
- If capacity grows geometrically, the cost is spread across many insertions.
- Insertion remains $O(1)$ amortized / expected

Hash Tables Performance

Operation	Expected	Worst case
Search	$O(1)$	$O(n)$
Insert/update	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$
len()	$O(1)$	$O(1)$
Iteration	$O(n)$	$O(n)$

Expected $O(1)$ $\neq$ guaranteed $O(1)$

Exercise

1. 9, 26, 50, 15, 2, 21, 36, 22, 32
2. 217, 209, 265, 226, 234, 201, 207, 223
3. 4322, 1334, 1471, 9679, 1989, 6171, 6173, 4199
4. 18, 29, 40, 63, 52, 33, 71, 25, 36
5. 22, 33, 14, 97, 89, 67, 61, 49

Given the hash function $h(k) = k \bmod 11$

- i. Insert the keys using
 - a. Separate chaining
 - b. Linear probing and count the probes
- ii. Compute the final load factor

Creative Question

You are given a list of words. Your task is to group the words that are anagrams of each other (words that contain the same letters but in a different order).

Example:

❑ Input: ["eat", "tea", "tan", "ate", "nat", "bat"]

❑ Output: [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]

Separate Chaining vs Linear Probing

❑ Separate chaining

- Easier to implement
- Performance degrades gracefully
- Clustering less sensitive to poorly-designed hash functions

❑ Linear probing

- Less wasted space
- Better cache performance

Hash Tables

- ❑ Hash tables are versatile and efficient in solving *real-world* applications like *lookup, grouping, and counting problems*.
- ❑ *Hashing* is a versatile tool that powers many modern systems and applications to ensure fast, reliable, and secure data management across various domains.
 - Secure password storage
 - Secure file integrity checking
 - Content delivery networks
 - Search engine indexing
 - Operating systems (e.g., LRU caching, where the LRU element is removed when the cache reaches capacity)